

Module 0: Environment, Notation, and Numerical Hygiene

1. Introduction

Welcome to the **XVA Calculus Visual Lab**. This suite of notebooks is designed to bridge the gap between abstract mathematical theory and practical, visual intuition for **X-Value Adjustments** — the pricing corrections that banks apply to derivatives to account for counterparty credit risk, funding costs, margin requirements, and regulatory capital.

In this first module, we establish our mathematical playground: the notation, the objects, and the code structure we will use throughout the course.

2. Mathematical Foundations (Detailed)

2.1 The Probability Space $(\Omega, \mathcal{F}, \mathbb{P})$

Stochastic calculus takes place on a probability space. This is a triple $(\Omega, \mathcal{F}, \mathbb{P})$ where:

1. **Sample Space Ω** : The set of all possible outcomes. In XVA, you can think of $\omega \in \Omega$ as a complete "scenario" — one realization of interest rates, FX rates, and default events for the entire trade lifetime.
 - *Intuition*: One Monte Carlo path of a swap portfolio is one ω .
2. **Sigma-Algebra $\mathcal{F}$** : The collection of all "measurable" events. An event $A \in \mathcal{F}$ is a subset of Ω for which we can assign a probability.
 - *Properties*: $\Omega \in \mathcal{F}$, closed under complements and countable unions.
3. **Probability Measure $\mathbb{P}$** : A function $\mathbb{P} : \mathcal{F} \rightarrow [0, 1]$ such that $\mathbb{P}(\Omega) = 1$ and it is countably additive.

2.2 SDEs and the Euler-Maruyama Scheme

Asset prices, interest rates, and hazard rates are modeled as stochastic processes satisfying:

$$dX_t = \mu(t, X_t) dt + \sigma(t, X_t) dW_t$$

To simulate SDEs on a computer, we divide $[0, T]$ into N steps of length $\Delta t = T/N$ and apply the Euler-Maruyama scheme:

$$X_{t_{i+1}} = X_{t_i} + \mu(t_i, X_{t_i}) \Delta t + \sigma(t_i, X_{t_i}) \sqrt{\Delta t} Z_i, \quad Z_i \sim \mathcal{N}(0, 1)$$

2.3 XVA Notation Reference

Throughout this course, we maintain the following conventions:

- $D(t, T) = e^{-r(T-t)}$: Discount factor from t to T .
- τ : The random default time of a counterparty.
- λ_t : Default intensity (hazard rate) — instantaneous default probability.
- $\mathbb{E}\mathbb{E}(t) = \mathbb{E}[V(t)^+]$: Expected Exposure (average positive MtM).
- $\text{PFE}_\alpha(t)$: Potential Future Exposure (α -quantile of positive exposure).
- R : Recovery rate; $\text{LGD} = 1 - R$: Loss Given Default.



Desk Reality: Why XVA Matters

Before the 2008 financial crisis, banks priced derivatives as if counterparties never default and funding is free. The collapse of Lehman Brothers caused $\sim \$5.2$ billion in CVA losses across the industry in a single quarter. Today, every major bank runs a dedicated **XVA desk** that centrally manages CVA, DVA, FVA, MVA, and KVA. These adjustments can represent 5-15% of the economic value of large OTC portfolios.

```
In [ ]: # --- PRESET DESK2QUANT BRAND STYLING ---
import matplotlib.pyplot as plt
plt.style.use('seaborn-v0_8-darkgrid')
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['font.size'] = 11
plt.rcParams['axes.labelsize'] = 12
plt.rcParams['axes.titlesize'] = 14
plt.rcParams['xtick.labelsize'] = 10
plt.rcParams['ytick.labelsize'] = 10
plt.rcParams['text.color'] = '#06275b'
plt.rcParams['axes.labelcolor'] = '#06275b'
plt.rcParams['xtick.color'] = '#06275b'
plt.rcParams['ytick.color'] = '#06275b'
plt.rcParams['grid.color'] = '#e2e8f0'
# Brand Colors Constants:
# D2Q_NAVY = '#06275b'
# D2Q_TEAL = '#08a9a2'
# D2Q_CRIMSON = '#f43f5e'
# -----

import numpy as np
import matplotlib.pyplot as plt
import ipywidgets as widgets
from IPython.display import display
from scipy.stats import norm
import QuantLib as ql
import time

rng = np.random.default_rng(42)

# --- Interactive: Law of Large Numbers ---
def law_of_large_numbers(N=100, M=5):
```

```

flips = rng.integers(0, 2, size=(M, N))
cum_heads = np.cumsum(flips, axis=1)
indices = np.arange(1, N+1)
running_avg = cum_heads / indices

plt.figure(figsize=(10, 5))
for i in range(M):
    plt.plot(indices, running_avg[i], alpha=0.6, linewidth=1.5)
plt.axhline(0.5, color='black', linestyle='--', linewidth=2, label='Theoretical')
plt.ylim(0, 1)
plt.title('Convergence of Frequency to Probability', color='#06275b', fontweight='bold')
plt.xlabel('Number of Flips $n$')
plt.ylabel('Proportion of Heads')
plt.legend()
plt.show()

widgets.interact(law_of_large_numbers,
                  N=widgets.IntSlider(min=10, max=2000, step=10, value=100, description='N'),
                  M=widgets.IntSlider(min=1, max=20, step=1, value=5, description='M'))

```

2.4 Processes & Filtrations (Analogy: The Book of History)

A stochastic process $X = \{X_t\}_{t \geq 0}$ is a family of random variables.

The Filtration $\mathbb{F} = \{\mathcal{F}_t\}_{t \geq 0}$ is the flow of information. Imagine a book where every page is a day in the market.

- $\mathcal{F}_0$: The book is closed. You know nothing except today's price.
- $\mathcal{F}_t$: You have read up to page t . You know everything that happened *before* t , but nothing after.
- $\mathcal{F}_T$: You have finished the book. The "randomness" is resolved.

Adaptedness: A process X_t is **adapted** if its value at time t can be determined just by reading the book up to page t . You don't need to peek at the ending.

3. Computational Setup & Hygiene

3.1 Seed Control: Alternate Universes

Good "numerical hygiene" is critical. In stochastic simulations, **Random Seeds** are your anchor. Without them, you cannot debug, because every run is different.

Use the slider to change the `seed`. Notice:

1. Same seed $\rightarrow$ Same path (Reproducibility).
2. Different seed $\rightarrow$ Completely different path.

```

In [ ]: def demonstrate_seed(seed=42):
        rng_local = np.random.default_rng(seed)

```

```

N = 100
paths = np.cumsum(rng_local.standard_normal((3, N)), axis=1)

plt.figure(figsize=(10, 5))
plt.plot(paths.T, marker='o', alpha=0.7, markersize=2)
plt.title(f'Random Paths Generated with Seed {seed}', color='#06275b', fontweight='bold')
plt.xlabel('Time Step')
plt.ylabel('Value')
plt.legend([f'Path {i}' for i in range(3)])
plt.show()

widgets.interact(demonstrate_seed,
                  seed=widgets.IntSlider(min=0, max=100, step=1, value=42, description='Seed'))

```

3.2 Vectorization: The Speed of Light

XVA simulations often involve 10,000+ paths and 100+ time steps for each of 1000+ trades. Python loops are too slow. We use **NumPy vectorization**.

Think of it like this:

- **Loop:** Asking 10,000 people one by one to roll a die.
- **Vectorization:** Asking 10,000 people to throw their dice all at once.

We will rely heavily on creating matrices of size (M, N) .

```

In [ ]: M, N = 5000, 1000
r0, theta, k, sigma_r = 0.03, 0.03, 0.5, 0.015
dt = 5.0 / N

# --- Loop implementation ---
start_loop = time.time()
r_loop = np.zeros((M, N + 1))
r_loop[:, 0] = r0
for j in range(M):
    for i in range(1, N + 1):
        z = np.random.standard_normal()
        r_loop[j, i] = r_loop[j, i-1] + k * (theta - r_loop[j, i-1]) * dt + sigma_r * z * np.sqrt(dt)
end_loop = time.time()

# --- Vectorized implementation ---
start_vec = time.time()
r_vec = np.zeros((M, N + 1))
r_vec[:, 0] = r0
Z = np.random.standard_normal((M, N))
for i in range(1, N + 1):
    r_vec[:, i] = r_vec[:, i-1] + k * (theta - r_vec[:, i-1]) * dt + sigma_r * Z[:, i-1] * np.sqrt(dt)
end_vec = time.time()

print(f'Loop took: {end_loop - start_loop:.4f} seconds')
print(f'Vectorized took: {end_vec - start_vec:.4f} seconds')
print(f'Speedup: {(end_loop - start_loop)/(end_vec - start_vec):.1f}x')

```

4. QuantLib: The Industry Standard Pricing Library

QuantLib is the open-source C++ library used across the industry for derivatives pricing. We use the Python bindings (`QuantLib-Python`). Let's verify it works and build our first yield curve.



Desk Reality: Curve Construction

Every morning, the rates desk constructs yield curves from market instruments (deposits, FRAs, swaps). QuantLib's `PiecewiseYieldCurve` bootstraps these instruments into a continuous discount curve. All XVA calculations depend on this curve.